Contenedores y tabla de direccionamiento directo

Escoger por el costo, y buscar sin recorrer

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Usar las operaciones propias de `list` y recorrer una `stack` o una `queue`, con el costo de cada una (OA2, OA3).
- Escoger los contenedores de un programa por las operaciones que repite y justificar la elección con su costo (OA1, OA5).
- Definir el TAD Diccionario e implementarlo con una tabla de direccionamiento directo, con el costo en tiempo y en espacio de cada operación (OA1, OA2, OA3).
- Reconocer cuándo el universo de claves hace inviable la tabla directa (OA1).

Contenido I

1 Lo que falta de los contenedores

2 Escoger por el costo: el taller

3 Para el juez

4 TAD tabla de direccionamiento directo

5 Ejercicios

Contenido I

1 Lo que falta de los contenedores

2 Escoger por el costo: el taller

3 Para el juez

4 TAD tabla de direccionamiento directo

5 Ejercicios

Los cuatro contenedores I

Contenedor	Por dentro	Lo barato
`list`	lista doblemente enlazada	insertar y borrar con el iterador, $\Theta(1)$
`deque`	bloques fijos y un mapa de punteros	los dos extremos y `d[i]`, $O(1)$
`stack`	un deque, por defecto	el último que entró, $O(1)$
`queue`	un deque, por defecto	el más viejo, $O(1)$

Pregunta

Hay que borrar todos los 8 de una `list`, o imprimir una `stack` sin perder lo que guarda. ¿Qué ofrece la biblioteca para eso, y qué cuesta?

Operaciones propias de `list` I

Problema

En `l = <3 3 8 1 1 1 6>` hay que dejar un solo elemento de cada grupo de repetidos seguidos, quitar los 8 e invertir el orden. Con un ciclo de `erase` se puede. ¿La lista trae algo propio?

Operaciones propias de `list` II

```
int main() {
  list<int> l = {3, 3, 8, 1, 1, 1, 6};
  l.unique();                  // borra los repetidos seguidos
  imprimir("unique", l);       // 3 8 1 6
  l.remove(8);                 // borra todos los 8
  imprimir("remove", l);       // 3 1 6
  l.reverse();                 // invierte los enlaces
  imprimir("reverse", l);      // 6 1 3

  list<int> a = {1, 4, 9};
  list<int> b = {2, 4, 7};
  a.merge(b);                  // las dos ordenadas: empalma los nodos
  ↪  de b
  imprimir("a", a);            // 1 2 4 4 7 9
  imprimir("b", b);            // vacia
  return 0;
}
```

Operaciones propias de `list` III

Salida

`unique: 3 8 1 6`, `remove: 3 1 6`, `reverse: 6 1 3`, `a: 1 2 4 4 7 9` y `b:` vacía.

Operaciones propias de `list` IV

Operación	Tiempo	Por qué
`l.unique()`	$\Theta(n)$	compara cada nodo con el anterior y desenlaza los repetidos
`l.remove(v)`	$\Theta(n)$	recorre una vez y desenlaza cada nodo con `v`
`l.reverse()`	$\Theta(n)$	cambia `anterior` por `siguiente` en cada nodo
`a.merge(b)`	$\Theta(n_a + n_b)$	las dos ordenadas: mezcla como merge sort, empalma los nodos de `b` y la deja vacía

- Ninguna copia datos ni pide nodos nuevos: espacio aparte $\Theta(1)$, y los iteradores a los nodos que quedan siguen valiendo.
- `unique` solo borra repetidos *seguidos*: para dejar uno de cada valor, primero `l.sort()`.

Recorrer una pila sin perderla I

Pregunta

`stack` y `queue` no tienen `begin()` ni `s[i]`. ¿Cómo se imprime una pila de la cima al fondo sin que quede vacía?

Recorrer una pila sin perderla II

```
// s llega por valor: es una copia, y la original no cambia
void imprimirPila(stack<int> s) {
  printf("pila, de la cima al fondo:");
  while (!s.empty()) {
    printf(" %d", s.top());
    s.pop();
  }
  printf("\n");
}
```

Salida, con 10, 20, 30 y 40 apilados y encolados

pila, de la cima al fondo: 40 30 20 10, cola, del frente al final: 10 20 30 40 y siguen con 4 y 4.

Recorrer una pila sin perderla III

- El parámetro llega por valor: la función vacía una copia. Copiar cuesta $\Theta(n)$ en tiempo y $\Theta(n)$ en espacio aparte, y vaciarla, otro $\Theta(n)$.
- Con `stack<int> &s`, la pila del que llama quedaría vacía.

Si el programa necesita recorrer

Recorrer con frecuencia es una operación que el contrato de la pila no ofrece. Ese programa pide un `deque` o un `vector`, que se recorren en $\Theta(n)$ sin copiar.

Contenido I

1 Lo que falta de los contenedores

2 Escoger por el costo: el taller

3 Para el juez

4 TAD tabla de direccionamiento directo

5 Ejercicios

Un taller con dos clases de pedido I

Problema

Un taller de bicicletas recibe pedidos de reparación rápida y de reparación normal. Cuando se desocupa, atiende la rápida más vieja; si no hay rápidas, la normal más vieja. Llegan 100 000 órdenes. ¿Qué contenedor guarda los pedidos?

¿Entendimos el problema? I

Entradas

Órdenes, una por línea: `R nombre` llega una rápida, `N nombre` llega una normal, `A` se atiende.

Salida

Por cada `A`, `atiende` y el nombre, o `nadie` si no hay pedidos.

Con `N Ana, N Beto, R Caro, A, A`

El primer `A` atiende a Caro, aunque llegó de última: es la única rápida. El segundo atiende a Ana, la normal más vieja.

Pregunta

Si después llegan `R Dani` y `N Eli` y se atiende cuatro veces, ¿qué se imprime?

¿Entendimos el problema? II

Orden	Esperan	Imprime
`R Dani`	Beto, Dani	
`N Eli`	Beto, Dani, Eli	
`A`	Beto, Eli	`atiende Dani`
`A`	Eli	`atiende Beto`
`A`		`atiende Eli`
`A`		`nadie`

Dani pasa delante de Beto, que esperaba desde antes: entre rápidas y normales manda la clase, y dentro de cada clase, el orden de llegada.

Primera idea: una sola fila I

Todos los pedidos en un deque, en orden de llegada. Atender busca la primera rápida con un iterador; si no hay, toma el frente.

```
// la primera rapida de la fila, o el frente si no hay rapidas
deque<Pedido>::iterator primeraRapida(deque<Pedido> &fila) {
  deque<Pedido>::iterator it = fila.begin();
  while (it != fila.end() && it->tipo != 'R') {
    ++it;
  }
  if (it == fila.end()) {
    it = fila.begin();
  }
  return it;
}
```

Primera idea: una sola fila II

```
// una sola fila: O(n) por atencion
void unaFila(int total) {
  deque<Pedido> fila;
  int i = 0;
  while (i < total) {
    if (tipos[i] != 'A') {
      fila.push_back(nuevo(i));
    } else if (fila.empty()) {
      printf("nadie\n");
    } else {
      deque<Pedido>::iterator it = primeraRapida(fila);
      printf("atiende %s\n", it->nombre);
      fila.erase(it);
    }
    i = i + 1;
  }
}
```

`nuevo(i)` arma el `Pedido` de la orden *i* con su clase y su nombre.

Primera idea: una sola fila III

Orden	Tiempo	Por qué
llega un pedido	$O(1)$	`push_back`
atender	$O(n)$	el ciclo camina hasta la primera rápida, y `erase` en medio corre el lado más corto

Con 50 000 normales esperando y una rápida al final, cada `A` camina 50 000 pedidos: las órdenes suman del orden de n^2.

Pregunta

El ciclo busca la primera rápida. ¿Hace falta buscarla, si se sabe de qué clase es cada pedido en el momento en que llega?

Dos colas I

No hace falta. Si cada pedido entra a la cola de su clase, la rápida más vieja es siempre el frente de `rapidas`.

```
// la rapida mas vieja; si no hay, la normal mas vieja. O(1)
void atender(queue<Pedido> &rapidas, queue<Pedido> &normales) {
  if (!rapidas.empty()) {
    printf("atiende %s\n", rapidas.front().nombre);
    rapidas.pop();
  } else if (!normales.empty()) {
    printf("atiende %s\n", normales.front().nombre);
    normales.pop();
  } else {
    printf("nadie\n");
  }
}
```

Dos colas II

```
// dos colas: O(1) por orden
void dosColas(int total) {
  queue<Pedido> rapidas;
  queue<Pedido> normales;
  int i = 0;
  while (i < total) {
    if (tipos[i] == 'R') {
      rapidas.push(nuevo(i));
    } else if (tipos[i] == 'N') {
      normales.push(nuevo(i));
    } else {
      atender(rapidas, normales);
    }
    i = i + 1;
  }
}
```

Dos colas III

Paso a paso con la entrada completa:

Orden	`rapidas`	`normales`	Imprime
`N Ana`		Ana	
`N Beto`		Ana, Beto	
`R Caro`	Caro	Ana, Beto	
`A`		Ana, Beto	`atiende Caro`
`A`		Beto	`atiende Ana`
`R Dani`	Dani	Beto	
`N Eli`	Dani	Beto, Eli	
`A`		Beto, Eli	`atiende Dani`
`A`		Eli	`atiende Beto`
`A`			`atiende Eli`
`A`			`nadie`

Las dos versiones imprimen lo mismo con esta entrada.

Dos colas IV

Orden	Una fila	Dos colas	Por qué
llega un pedido	$O(1)$	$O(1)$	`push` al final de su cola
atender	$O(n)$	$O(1)$	se mira el frente de `rapidas` y, si está vacía, el de `normales`
q órdenes	$O(q^2)$	$\Theta(q)$	q órdenes de $O(1)$

- Espacio: $\Theta(n)$ en las dos, un pedido por casilla. La segunda no gasta más memoria; reparte la misma en dos colas.
- La versión de una fila guardaba el orden de llegada de todos juntos, y el problema solo usa el orden dentro de cada clase.

Ejercicio en parejas: escoger y costear I

Para cada programa, escojan los contenedores y digan qué cuesta la operación que más se repite.

1. Un navegador con botones *atrás* y *adelante*. Visitar una página nueva borra lo que había adelante.
2. En una banda de empaque, cada caja sale por el frente; si una sale mal sellada, vuelve al frente para repetirla antes que las demás.
3. Se leen números hasta el fin de archivo y se imprimen al revés.
4. En una cocina los pedidos se preparan en orden de llegada, y una pantalla muestra siempre los cinco primeros.
5. En la lista de espera de un restaurante, cuando se libera una mesa grande se retiran todos los grupos de más de seis personas, estén donde estén.

Ejercicio en parejas: escoger y costear II

Lo que debe salir

1. Dos `stack`, `atras` y `adelante`: cada botón saca de una y apila en la otra, $O(1)$. Visitar apila en `atras` y deja vacía `adelante`, a costa de liberar lo que tenía.
2. `deque`: `pop_front` para sacar y `push_front` para devolver, $O(1)$. Una `queue` no deja entrar por el frente.
3. `stack`: n `push` y n `pop`, $\Theta(n)$ en total.
4. `deque`: entra por el final y sale por el frente en $O(1)$, y la pantalla lee `d[0]` a `d[4]` sin copiar. Con `queue` cada refresco copiaría la cola.
5. `list`: un recorrido con `erase` retira a todos en $\Theta(n)$. Con `vector` y un `erase` por grupo, cada borrado corre a los de atrás: del orden de n^2 en el peor caso.

Contenido I

1 Lo que falta de los contenedores

2 Escoger por el costo: el taller

3 Para el juez

4 TAD tabla de direccionamiento directo

5 Ejercicios

Para el juez: UVa 442, Matrix Chain Multiplication I

UVa 442, Matrix Chain Multiplication,
`https://onlinejudge.org/external/4/442.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=383`

Qué pide

Llegan $n \leq 26$ matrices, cada una con una letra mayúscula, sus filas y sus columnas. Después, hasta el fin de archivo, una expresión por línea con paréntesis completos, como `(A(BC))`. Multiplicar una matriz de $p \times q$ por una de $q \times r$ cuesta $p \cdot q \cdot r$ multiplicaciones. Por cada expresión se imprime el total, o `error` si dos matrices no se pueden multiplicar.

Entradas y salidas del ejemplo

Con A de 50×10, B de 10×20 y C de 20×5: `A` da 0, `(AA)` da `error`, `(A(BC))` da 3500 y `((AB)C)` da 15000.

Para el juez: UVa 442, Matrix Chain Multiplication II

Cómo atacarlo

- Las dimensiones van en un arreglo de 26 casillas: la de la letra c está en la casilla `c - 'A'`. Leerla cuesta $\Theta(1)$, sin buscar.
- La expresión se lee de izquierda a derecha con una `stack` de dimensiones. Una letra apila su matriz; un `)` saca dos, revisa que las columnas de la izquierda sean las filas de la derecha, suma $p \cdot q \cdot r$ y apila la matriz de $p \times r$ que resulta. El `(` no hace nada.
- Cada carácter cuesta $O(1)$: $\Theta(L)$ por expresión de largo L, y $O(L)$ de espacio en la pila.

Para el juez: UVa 442, Matrix Chain Multiplication III

Las trampas

- El primero que sale de la pila es la matriz de la *derecha*. Con el orden al revés, `(AB)` revisa las columnas de B contra las filas de A y da `error`.
- Una expresión de una sola letra no multiplica nada: imprime 0.
- `(AA)` es válida en la sintaxis y da `error` porque A no es cuadrada. Tras el primer error ya no se suma.
- El enunciado no acota las dimensiones: el total se acumula en `long long`.

Contenido I

1 Lo que falta de los contenedores

2 Escoger por el costo: el taller

3 Para el juez

4 TAD tabla de direccionamiento directo

5 Ejercicios

¿Quién entró al laboratorio? I

Problema

En la puerta de un laboratorio se registra a cada estudiante que entra, con un código entre 0 y 999 y su nombre. Durante el día llegan miles de preguntas: ¿entró el 412? Y a veces se retira un registro. ¿Cómo se responde cada pregunta sin revisar todos los registros?

¿Entendimos el problema? I

Entradas

Tres clases de orden: registrar un código con un nombre, preguntar por un código y retirar un código. Un código no se registra dos veces; si llega otra vez, cambia el nombre.

Salida

Por cada pregunta, el nombre registrado con ese código o `no esta`.

Con códigos de 0 a 9

Se registran 3 Ana, 7 Luis y 0 Eva. Se pregunta por el 7 y por el 5, se retira el 3 y se pregunta por el 3.

Pregunta

¿Qué se imprime en cada una de las tres preguntas?

¿Entendimos el problema? II

Orden	Registrados	Imprime
registrar 3 Ana	3 Ana	
registrar 7 Luis	3 Ana, 7 Luis	
registrar 0 Eva	3 Ana, 7 Luis, 0 Eva	
preguntar 7		`buscar(7): Luis`
preguntar 5		`buscar(5): no esta`
retirar 3	7 Luis, 0 Eva	
preguntar 3		`buscar(3): no esta`

El 3 estuvo registrado, pero se retiró antes de la pregunta.

Primera idea: una lista I

Los registros en una `list`, en orden de llegada. Preguntar camina desde el comienzo hasta encontrar el código.

```
// Theta(n) en el peor caso: camina hasta encontrar la clave o hasta
↪  el final
list<Registro>::iterator buscar(list<Registro> &l, int clave) {
  list<Registro>::iterator it = l.begin();
  while (it != l.end() && it->clave != clave) {
    ++it;
  }
  return it;
}
```



`buscar(5)`: tres pasos y no está

Primera idea: una lista II

Operación	Tiempo	Por qué
registrar	$\Theta(1)$	`push_back`, si no se revisa que el código ya esté
preguntar	$\Theta(n)$ en el peor caso	camina nodo a nodo; un código que no está recorre los n
retirar	$\Theta(n)$ en el peor caso	buscar el nodo; `erase` ya es $\Theta(1)$

Con 800 registrados y 10 000 preguntas por códigos que no están, se recorren 8 000 000 nodos. El espacio es $\Theta(n)$.

Pregunta

El código ya es un número entre 0 y 999. ¿Hace falta buscarlo, o puede decir él mismo dónde está guardado?

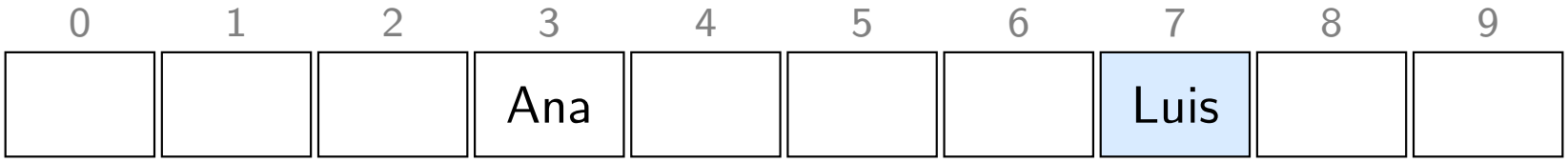
La clave dice la casilla I

Un arreglo con una casilla por cada código posible: el registro del código k va en la casilla k. Con códigos de 0 a 9, diez casillas, todas vacías al comenzar.

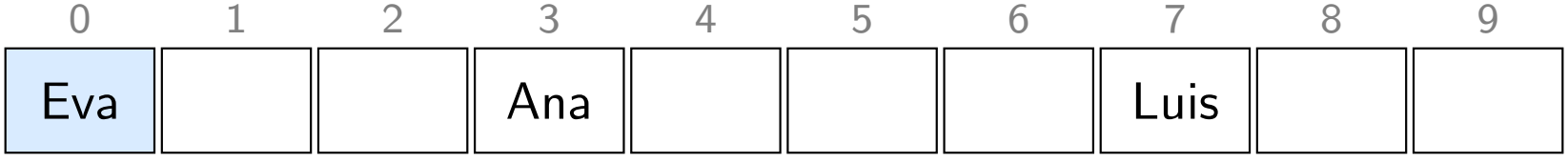


`insertar(3, Ana)`: la clave es 3, así que se escribe en la casilla 3. No se mira ninguna otra.

La clave dice la casilla II



`insertar(7, Luis)`: casilla 7.



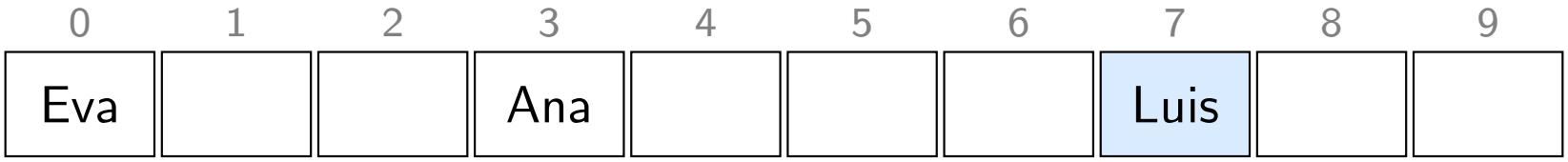
`insertar(0, Eva)`: casilla 0. El orden de llegada no importa; cada clave tiene su sitio.

La clave dice la casilla III

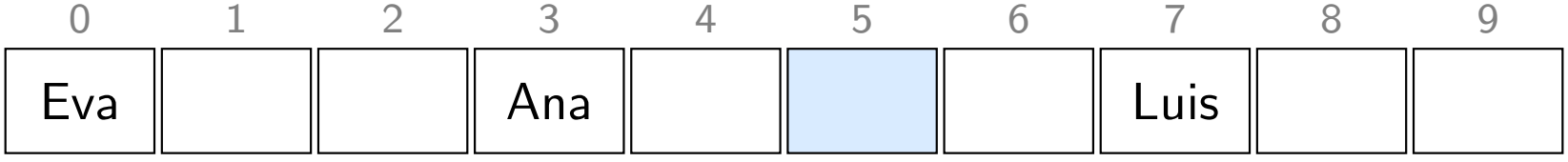
Pregunta

¿Cuántas casillas se leen para responder `buscar(7)`? ¿Y `buscar(5)`?

La clave dice la casilla IV



`buscar(7)`: una casilla, la 7, y ahí está Luis.



`buscar(5)`: también una sola. La casilla 5 está vacía, así que el 5 no está registrado, y no hace falta mirar las demás.

La clave dice la casilla V



`eliminar(3)`: se vacía la casilla 3. Después, `buscar(3)` lee esa casilla y dice que no está.

Ninguna operación miró más de una casilla, y ninguna dependió de cuántos registros había.

El TAD Diccionario I

Diccionario (CLRS, introducción del capítulo 11)

Un conjunto dinámico de elementos, cada uno con una *clave* y datos que la acompañan, con tres operaciones:

- `insertar(x)`: agrega el elemento `x`.
- `buscar(k)`: devuelve el elemento de clave `k`, o `NULL` si no hay.
- `eliminar(k)`: quita el elemento de clave `k`.

Tabla de direccionamiento directo (CLRS, §11.1)

Las claves salen de un universo $U = \{0, 1, \ldots, m-1\}$, con m no muy grande, y dos elementos no tienen la misma clave. La tabla es un arreglo $T[0..m-1]$: la casilla k apunta al elemento de clave k, o vale `NULL` si no hay ninguno.

El TAD Diccionario II

En el laboratorio

El universo son los códigos de 0 a 999, $m = 1000$; el elemento es un registro con código y nombre, y la clave es el código.

El diseño en pseudocódigo I

Las tres operaciones (CLRS, §11.1)

```
buscar(T, k)
   devolver T[k]

insertar(T, x)
   T[x.clave] ← x

eliminar(T, k)
   T[k] ← NULL
```

Cada una calcula una dirección y lee o escribe una casilla. No hay ciclo ni comparación de claves.

El diseño en pseudocódigo II

Lo que la tabla supone

- Toda clave está entre 0 y $m - 1$. Una clave fuera de ese rango escribe fuera del arreglo.
- Las claves no se repiten: con dos registros de clave 3, el segundo ocupa la casilla del primero.

TablaDirecta en C++ I

Un arreglo de punteros a `Registro` pedido con `new`, todos en `NULL` al comenzar.

```
struct Registro {
  int clave;
  char nombre[20];
};

class TablaDirecta {
private:
  Registro **T;       // T[k] apunta al registro de clave k, o vale
  ↪  NULL
  int m;              // tamano del universo de claves

public:
```

TablaDirecta en C++ II

```
// Theta(m): todas las casillas empiezan vacias
TablaDirecta(int tam) {
  m = tam;
  T = new Registro*[m];
  int k = 0;
  while (k < m) {
    T[k] = NULL;
    k = k + 1;
  }
}
```

TablaDirecta en C++ III

```
// Theta(m): se revisan todas las casillas
~TablaDirecta() {
  int k = 0;
  while (k < m) {
    delete T[k];
    k = k + 1;
  }
  delete[] T;
}

// exige 0 <= clave < m; si la clave ya estaba, cambia el nombre
void insertar(int clave, const char *nombre) {
  assert(clave >= 0 && clave < m);
  if (T[clave] == NULL) {
    T[clave] = new Registro;
    T[clave]->clave = clave;
  }
  strcpy(T[clave]->nombre, nombre);
}
```

TablaDirecta en C++ IV

```
// exige 0 <= clave < m; devuelve NULL si no hay registro
Registro *buscar(int clave) {
  assert(clave >= 0 && clave < m);
  return T[clave];
}

// exige 0 <= clave < m
void eliminar(int clave) {
  assert(clave >= 0 && clave < m);
  delete T[clave];
  T[clave] = NULL;
}
```

- La tabla es dueña de los registros: `insertar` los pide con `new` y `eliminar` los libera con `delete`. `delete` sobre `NULL` no hace nada, así que eliminar un código que no está es válido.
- `buscar` devuelve el puntero de la casilla: o el registro, o `NULL`.

El laboratorio con la tabla I

```
int main() {
  TablaDirecta t(10);
  t.insertar(3, "Ana");
  t.insertar(7, "Luis");
  t.insertar(0, "Eva");
  t.imprimir();
  consultar(t, 7);
  consultar(t, 5);
  t.eliminar(3);
  consultar(t, 3);
  t.imprimir();
  return 0;
}
```

Los dos `imprimir()` escriben `0:Eva 1:- 2:- 3:Ana 4:- 5:- 6:- 7:Luis 8:- 9:-` y `0:Eva 1:- 2:- 3:- 4:- 5:- 6:- 7:Luis 8:- 9:-`.

El laboratorio con la tabla II

Operación	Casilla	Leídas	Salida
`insertar(3, Ana)`	3	1	
`insertar(7, Luis)`	7	1	
`insertar(0, Eva)`	0	1	
`imprimir()`	todas	10	la tabla, con Ana en la 3
`consultar(7)`	7	1	`buscar(7): Luis`
`consultar(5)`	5	1	`buscar(5): no esta`
`eliminar(3)`	3	1	
`consultar(3)`	3	1	`buscar(3): no esta`
`imprimir()`	todas	10	la tabla, con la 3 vacía

La salida es la misma que la de la lista. Lo que cambia es cuántas casillas se miran: una por operación, salvo `imprimir`, que recorre las m.

Lo que cobra la tabla I

Operación	Tiempo	Por qué
crear la tabla	$\Theta(m)$	cada casilla empieza en `NULL`
`insertar(k, nombre)`	$\Theta(1)$	la clave es la dirección: una casilla y, si está vacía, un `new`
`buscar(k)`	$\Theta(1)$	se lee $T[k]$
`eliminar(k)`	$\Theta(1)$	un `delete` y $T[k] \leftarrow$ `NULL`
`imprimir`, destruir	$\Theta(m)$	recorren todas las casillas, llenas o vacías

Espacio

$\Theta(m)$ punteros, uno por clave *posible*, más $\Theta(n)$ registros, uno por clave guardada. Como $n \leq m$, el total es $\Theta(m) = \Theta(|U|)$: lo fija el universo, no lo que se guarda. Con $m = 1000$ y punteros de 8 bytes, la tabla vacía ya ocupa 8 000 bytes.

Guardar un número en la casilla I

Problema

Se tienen las notas de un curso en décimas, de 0 a 50: 35 es 3,5. ¿Cuántas veces aparece cada nota?

Cuando el elemento es solo su clave, la casilla no necesita un puntero: basta un contador. La nota es la casilla.

Guardar un número en la casilla II

```
const int M = 51;     // notas de 0 a 50

// cuenta[k] termina con las veces que aparecio la nota k
void contar(int cuenta[]) {
  int k = 0;
  while (k < M) {
    cuenta[k] = 0;
    k = k + 1;
  }
  int nota;
  while (scanf("%d", &nota) == 1) {
    cuenta[nota] = cuenta[nota] + 1;      // la nota es la casilla
  }
}
```

Guardar un número en la casilla III

```
void imprimir(int cuenta[]) {
  int k = 0;
  while (k < M) {
    if (cuenta[k] > 0) {
      printf("%d,%d: %d\n", k / 10, k % 10, cuenta[k]);
    }
    k = k + 1;
  }
}

int main() {
  int cuenta[M];
  contar(cuenta);
  imprimir(cuenta);
  return 0;
}
```

Guardar un número en la casilla IV

Con `35 42 30 35 50 42 35 18`

Imprime `1,8: 1, 3,0: 1, 3,5: 3, 4,2: 2` y `5,0: 1`.

- Contar cuesta $\Theta(1)$ por nota y $\Theta(n)$ las n; iniciar e imprimir, $\Theta(m)$ con $m = 51$. Total $\Theta(n + m)$, en espacio aparte $\Theta(m)$.
- Con una lista de pares (nota, cuenta), cada nota buscaría su par: $O(n \cdot m)$ en el peor caso.
- Las notas salen en orden sin ordenar nada, porque las casillas ya están en orden.

La clave fuera del universo

Una nota de 51 o de -1 escribe fuera de `cuenta`. La tabla directa confía en que toda clave está en U; si la entrada no lo garantiza, se revisa antes de escribir.

El límite: un universo grande I

Pregunta

La puerta del laboratorio registra ahora por número de documento, de hasta 10 dígitos. En un día entran 40 personas. ¿Cuánto ocupa la tabla?

El límite: un universo grande II

```c
int main() {
  long long puntero = sizeof(Registro *);
  long long m1 = 1000;
  long long m2 = 10000000000;      // codigos de 10 digitos
  printf("puntero: %lld bytes\n", puntero);
  printf("m = %lld: %lld bytes\n", m1, m1 * puntero);
  printf("m = %lld: %lld bytes\n", m2, m2 * puntero);
  return 0;
}
```

Salida

puntero: 8 bytes, m = 1000: 8000 bytes y m = 10000000000: 80000000000 bytes.

El límite: un universo grande III

- Ochenta mil millones de bytes, 80 GB, para guardar 40 registros: se usarían 40 casillas de 10^{10}.
- Crear la tabla cuesta $\Theta(m)$: poner 10^{10} casillas en `NULL` antes de la primera pregunta.

	Lista	Tabla directa
buscar, eliminar	$\Theta(n)$	$\Theta(1)$
insertar	$\Theta(1)$	$\Theta(1)$
espacio	$\Theta(n)$	$\Theta(\|U\|)$
sirve cuando	n es pequeño	$\|U\|$ es pequeño

El límite: un universo grande IV

Pregunta abierta

Con 100 casillas y los dos últimos dígitos del documento como casilla, la tabla ocupa 800 bytes. ¿Qué pasa cuando llegan dos documentos que terminan en los mismos dos dígitos?

Contenido I

1 Lo que falta de los contenedores

2 Escoger por el costo: el taller

3 Para el juez

4 TAD tabla de direccionamiento directo

5 Ejercicios

Ejercicio en parejas: ¿hay repetidos? I

Dado un arreglo de n enteros, todos entre 0 y 999, decir si algún valor aparece dos veces.

Las preguntas

1. Con $\langle 412, 87, 903, 15, 87, 230 \rangle$ y con $\langle 5, 999, 0, 314 \rangle$, ¿qué se responde?
2. Escriban la versión que compara cada par. ¿Cuánto cuesta?
3. ¿Qué guarda la casilla v de una tabla de 1000 casillas para responder en un solo recorrido?

Ejercicio en parejas: ¿hay repetidos? II

```
// compara cada par: Theta(n^2) en el peor caso
bool repetidoDirecto(int a[], int n) {
  bool hay = false;
  int i = 0;
  while (i < n && !hay) {
    int j = i + 1;
    while (j < n && !hay) {
      if (a[i] == a[j]) {
        hay = true;
      }
      j = j + 1;
    }
    i = i + 1;
  }
  return hay;
}
```

Ejercicio en parejas: ¿hay repetidos? III

- El primer arreglo tiene el 87 dos veces: `si`. El segundo, `no`.
- Sin repetidos compara los $n(n-1)/2$ pares: $\Theta(n^2)$ en el peor caso, espacio aparte $\Theta(1)$.

Ejercicio en parejas: ¿hay repetidos? IV

```
// una bandera por valor posible: O(n + m)
bool repetidoTabla(int a[], int n) {
  bool visto[M];
  int k = 0;
  while (k < M) {
    visto[k] = false;
    k = k + 1;
  }
  bool hay = false;
  int i = 0;
  while (i < n && !hay) {
    if (visto[a[i]]) {
      hay = true;
    }
    visto[a[i]] = true;
    i = i + 1;
  }
  return hay;
}
```

Ejercicio en parejas: ¿hay repetidos? V

- La casilla v guarda una bandera: si ya apareció el valor v. Iniciar cuesta $\Theta(m)$ y recorrer, $O(n)$: en total $O(n+m)$, con $m = 1000$, y $\Theta(m)$ de espacio aparte.
- Las dos versiones imprimen `a: si si` y `b: no no`. Con $n = 100\,000$, la directa compara del orden de $5 \cdot 10^9$ pares y la tabla hace del orden de 10^5 pasos.

Ejercicio en parejas: la clave más grande I

Pregunta

Una `TablaDirecta` de m casillas guarda n registros. ¿Cómo se encuentra el registro de clave más grande, y cuánto cuesta en el peor caso?

Ejercicio en parejas: la clave más grande II

Lo que debe salir

Se recorre desde la casilla $m - 1$ hacia abajo y se para en la primera que no esté vacía. Si el único registro tiene clave 0, se miran las m casillas: $\Theta(m)$ en el peor caso, aunque n sea 1. La tabla no guarda orden entre las casillas llenas; el máximo cuesta lo que cuesta recorrer el universo.

- En la lista del laboratorio, el máximo recorre los n nodos: $\Theta(n)$. Aquí la lista gana, porque $n \leq m$.
- Guardar aparte la clave más grande la deja en $\Theta(1)$ al insertar, pero al eliminar justo esa clave hay que volver a buscar.

Propuestos para la casa I

1. **Conjunto de banderas.** Implemente un conjunto de enteros de 0 a $m-1$ con un arreglo de `bool`: `agregar`, `quitar`, `pertenece`, `unir` e `intersecar`. Dé el costo de cada una.
2. **Ordenar contando.** Se leen n notas en décimas, de 0 a 50. Imprímalas ordenadas, con repeticiones, usando la tabla de contadores. Compare el costo con el de merge sort.
3. **Claves repetidas.** En el laboratorio, ahora un mismo código puede entrar varias veces y hay que guardar todas las horas de entrada. ¿Qué guarda cada casilla? ¿Cuánto cuesta cada operación?
4. **Pilas que no se pierden.** Escriba una función que reciba una `stack<int>` por referencia, imprima del fondo a la cima y la deje como estaba.
5. **El taller con tres clases.** Agregue las reparaciones urgentes, que pasan delante de las rápidas. ¿Cuántas colas hacen falta? ¿Cambia el costo de atender?

Pistas para los propuestos I

1. Las tres primeras leen o escriben una casilla, $\Theta(1)$; unir e intersecar recorren las m casillas con `||` y `&&`, $\Theta(m)$.
2. Se cuenta como en `contar_notas` y después se recorre la tabla imprimiendo cada nota k tantas veces como diga `cuenta[k]`: $\Theta(n+m)$. Merge sort cuesta $\Theta(n \log n)$, y no necesita saber que las notas van de 0 a 50.
3. Cada casilla apunta a una lista con las horas de ese código. Insertar al frente de la lista sigue en $\Theta(1)$; borrar una hora en particular camina la lista de su código.
4. Pase los elementos a una pila auxiliar: salen al revés, del fondo a la cima. Imprima mientras los devuelve. $\Theta(n)$ en tiempo y $\Theta(n)$ en espacio aparte, la pila auxiliar.
5. Una cola por clase, revisadas en orden de prioridad. Con un número fijo de clases, atender sigue en $O(1)$.

Un reto para cerrar I

Los códigos del laboratorio ahora van de 1000 a 1999. Escriba `buscar` e `insertar` con una tabla de 1000 casillas, sin desperdiciar ninguna. Después, los códigos válidos son solo los pares entre 0 y 1998. ¿Cuántas casillas hacen falta, y qué cuenta convierte un código en su casilla?

Resumen I

- `unique`, `remove`, `reverse` y `merge` de `list` cuestan $\Theta(n)$ y no copian datos. Recorrer una `stack` o una queue sin perderla cuesta una copia, $\Theta(n)$.
- Escoger contenedor es mirar la operación que más se repite: en el taller, dos colas dejan atender en $O(1)$, y una sola fila cuesta $O(n)$.
- El TAD Diccionario tiene `insertar`, `buscar` y `eliminar`. Con claves en $U = \{0, \ldots, m-1\}$, la tabla de direccionamiento directo las hace en $\Theta(1)$: la clave es la casilla.
- La tabla ocupa $\Theta(|U|)$ aunque guarde pocos elementos, y crearla o recorrerla cuesta $\Theta(m)$.

Lo que sigue

Tablas hash: una tabla de pocas casillas para un universo enorme, con una función que convierte la clave en casilla, y lo que se hace cuando dos claves caen en la misma.

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 11, tablas hash: introducción, operaciones de diccionario; §11.1, tablas de direccionamiento directo.



cppreference.com. Containers library: `https://en.cppreference.com/w/cpp/container/list`, `https://en.cppreference.com/w/cpp/container/stack`, `https://en.cppreference.com/w/cpp/container/queue`.



UVa 442, Matrix Chain Multiplication. `https://onlinejudge.org/external/4/442.pdf`.